

QuickCart – End-to-End DevOps Project

1. Project Overview

QuickCart is a Java Spring Boot-based e-commerce backend application developed to demonstrate an end-to-end DevOps workflow.

The project implements automated application building, testing, code quality analysis, container security scanning, container image management, and Kubernetes deployment on Microsoft Azure.

The complete CI/CD workflow is:

GitHub → Jenkins → Maven → SonarQube → Trivy → Azure Container Registry → Azure Kubernetes Service

The application is exposed through an Azure LoadBalancer and can be accessed through a public endpoint.

Key Objectives

- Develop a REST-based Spring Boot application.
- Implement automated build and testing using Maven.
- Automate the CI/CD process using Jenkins.
- Perform static code analysis using SonarQube.
- Scan Docker images for vulnerabilities using Trivy.
- Store container images in Azure Container Registry (ACR).
- Deploy the application to Azure Kubernetes Service (AKS).
- Expose the application using a Kubernetes LoadBalancer.
- Verify the application through a public API endpoint.

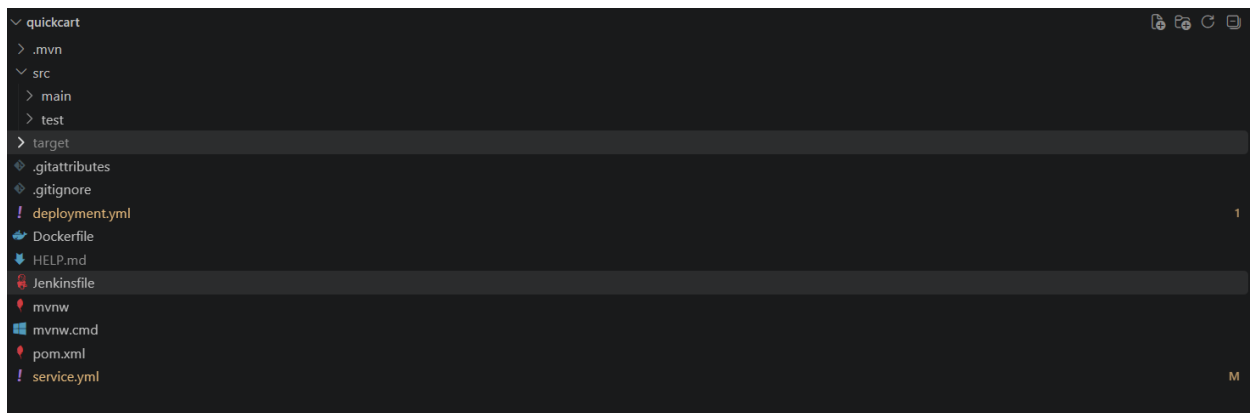
2. Project Structure

The QuickCart project follows a standard Maven/Spring Boot structure.

Important Files and Directories

File / Directory	Purpose
src/main	Contains the application source code
src/test	Contains automated tests
pom.xml	Maven project configuration and dependencies
Dockerfile	Defines how the application Docker image is built
Jenkinsfile	Defines the Jenkins CI/CD pipeline
deployment.yml	Defines the Kubernetes Deployment
service.yml	Defines the Kubernetes Service
.gitignore	Specifies files ignored by Git

Figure 1: QuickCart Project Structure



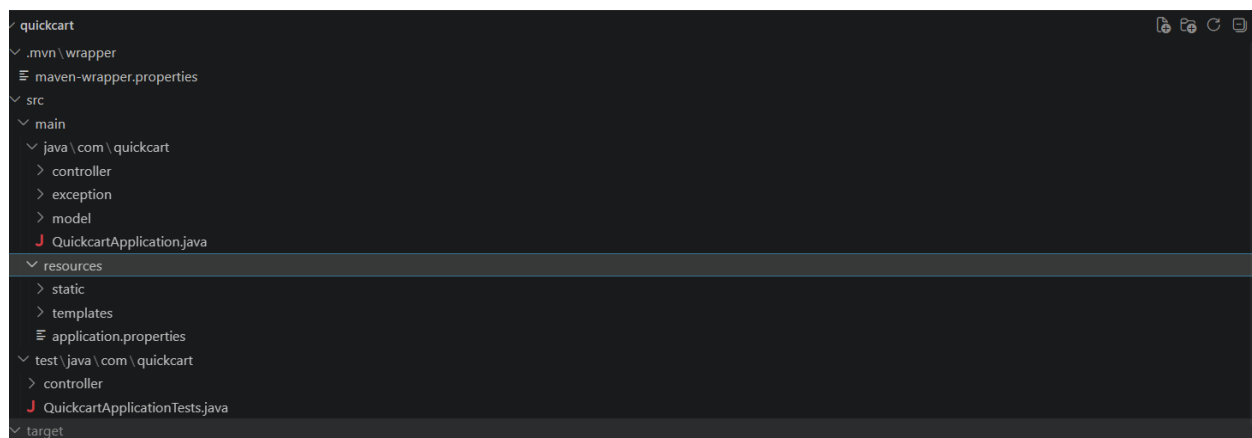
3. Application Structure

QuickCart is implemented as a **Java Spring Boot REST API**. The application follows a simple layered structure to separate API controllers, data models, exception handling, and application configuration.

Main Components

- **Controller** – Handles HTTP requests and exposes the QuickCart REST APIs.
- **Model** – Contains the data structures used by the application, such as products and orders.
- **Exception Handler** – Provides centralized handling for validation errors and returns appropriate HTTP responses.
- **Application Class** – Serves as the entry point for the Spring Boot application.
- **Resources** – Contains application configuration such as `application.properties`.
- **Test** – Contains automated tests for validating application functionality.

Figure 2: QuickCart Application Source Structure



4. API Testing and Application Verification

The QuickCart application exposes REST APIs for managing products and orders.

The deployed application was tested through the public Azure Kubernetes Service endpoint. The Product API was accessed using:

```
GET /api/products
```

The API successfully returned the available products, confirming that:

- The QuickCart application is running successfully on AKS.
- The Kubernetes Deployment is operational.
- The Kubernetes LoadBalancer is exposing the application externally.
- The application is accessible through the public Azure endpoint.

Figure 3: QuickCart Product API Response from Azure AKS

```
PS C:\Users\Abhisheik> Invoke-RestMethod http://4.247.239.65:8081/api/products

id name                price
-- ----
1 Wireless Headphones 2499.0
2 Mechanical Keyboard 3499.0
3 Gaming Mouse        1499.0

PS C:\Users\Abhisheik> |
```

5. Dockerization

To package the QuickCart application consistently across different environments, the Spring Boot application was containerized using Docker.

The Dockerfile uses **Eclipse Temurin Java 21 JRE** as the base image. The generated Spring Boot JAR file is copied into the container, and the application is configured to run on port **8081**.

Dockerfile

```
FROM eclipse-temurin:21-jre
```

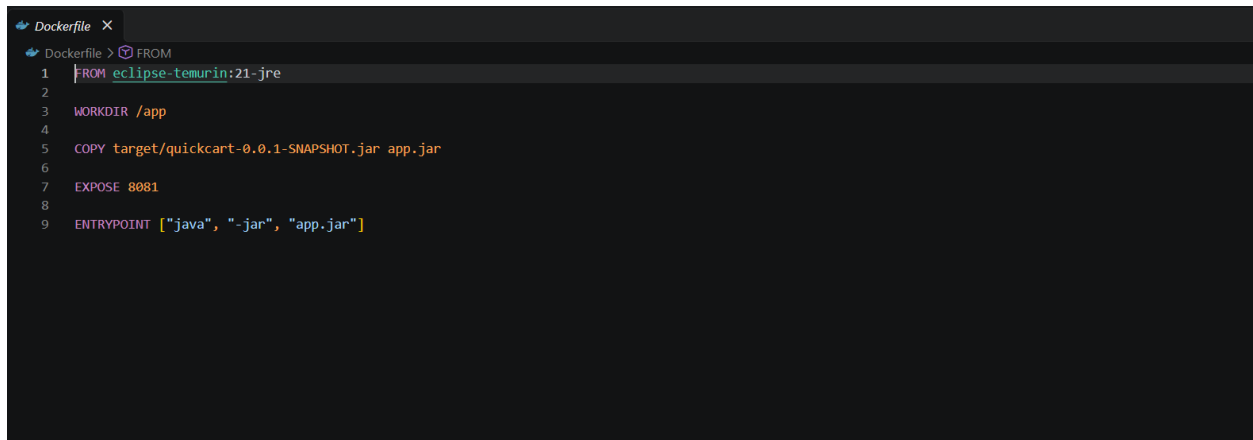
```
WORKDIR /app
```

```
COPY target/quickcart-0.0.1-SNAPSHOT.jar app.jar
```

```
EXPOSE 8081
```

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Figure 4: QuickCart Dockerfile



```
Dockerfile
1 FROM eclipse-temurin:21-jre
2
3 WORKDIR /app
4
5 COPY target/quickcart-0.0.1-SNAPSHOT.jar app.jar
6
7 EXPOSE 8081
8
9 ENTRYPOINT ["java", "-jar", "app.jar"]
```

6. CI/CD Pipeline with Jenkins

Jenkins was used to automate the complete CI/CD workflow for the QuickCart application.

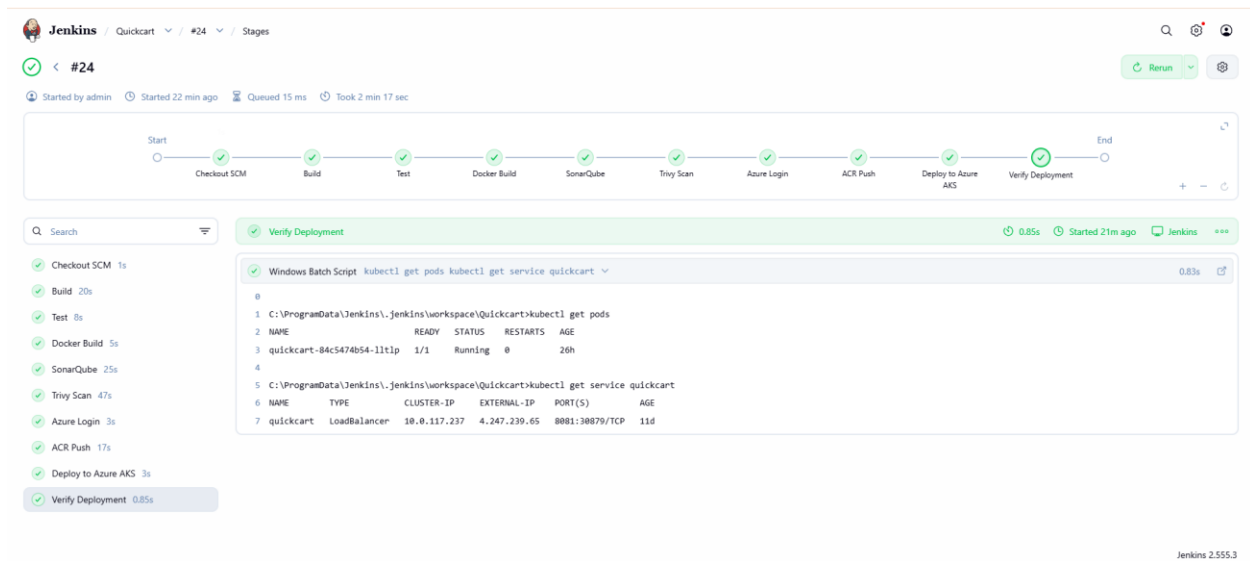
The pipeline performs the following stages:

1. **Checkout SCM** – Retrieves the latest source code from GitHub.
2. **Build** – Builds the Spring Boot application using Maven.
3. **Test** – Executes the automated test suite.
4. **Docker Build** – Creates the QuickCart Docker image.
5. **SonarQube** – Performs static code quality analysis.
6. **Trivy Scan** – Scans the Docker image for HIGH and CRITICAL vulnerabilities.
7. **Azure Login** – Authenticates Jenkins with Microsoft Azure using a Service Principal.
8. **ACR Push** – Pushes the validated Docker image to Azure Container Registry.
9. **Deploy to Azure AKS** – Deploys the application to the Kubernetes cluster.
10. **Verify Deployment** – Verifies the running Kubernetes pod and LoadBalancer service.

The successful pipeline demonstrates that the application can move from source code to a running Azure Kubernetes deployment through an automated CI/CD process.

The final verification confirms that the QuickCart pod is running with 1/1 containers ready and that the Kubernetes service is exposed through an Azure LoadBalancer.

Figure 5: Successful QuickCart Jenkins CI/CD Pipeline



7. Code Quality Analysis with SonarQube

SonarQube was integrated into the Jenkins CI/CD pipeline to perform automated static code analysis on the QuickCart application.

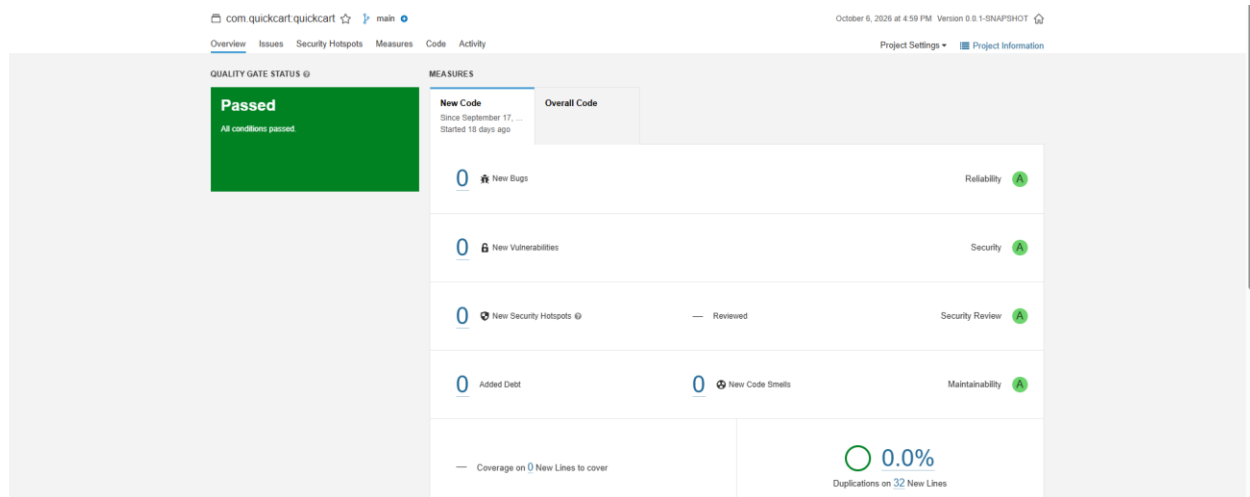
The Jenkins pipeline executes the SonarQube analysis and checks the configured Quality Gate. The pipeline is allowed to continue only when the Quality Gate passes.

The QuickCart project achieved a **Passed Quality Gate** with the following results:

- **New Bugs:** 0
- **New Vulnerabilities:** 0
- **Security Hotspots:** 0
- **New Code Smells:** 0
- **Reliability Rating:** A
- **Security Rating:** A
- **Maintainability Rating:** A

Figure 6: QuickCart SonarQube Quality Gate Results

This confirms that the application successfully passed the configured code-quality and security checks before proceeding with the deployment stages.



8. Container Security Scanning with Trivy

Trivy was integrated into the Jenkins CI/CD pipeline to scan the QuickCart Docker image for security vulnerabilities.

The pipeline checks for **HIGH and CRITICAL severity vulnerabilities** before allowing the image to proceed to Azure Container Registry.

The final scan reported:

- **Docker image vulnerabilities:** 0
- **Application JAR vulnerabilities:** 0
- **Gobinary vulnerabilities:** 0

The scan completed successfully with no security findings detected.

Figure 7: Trivy Container Security Scan Results

16 Report Summary

17

18	Target	Type	Vulnerabilities	Secrets
19				
20				
21	quickcartacr912.azurecr.io/quickcart:latest (ubuntu 26.04)	ubuntu	0	-
22				
23	app/app.jar	jar	0	-
24				
25	usr/bin/pebble	gobinary	0	-
26				

27 Legend:

28 - '-': Not scanned

29 - '0': Clean (no security findings detected)

30

31

32 📢 Notices:

33 - Version 0.75.0 of Trivy is now available, current version is 0.72.0

34

35 To suppress version checks, run Trivy scans with the --skip-version-check flag

36

This security check helps ensure that vulnerable container images are not pushed to the container registry or deployed to the Kubernetes environment.

9. Azure Container Registry

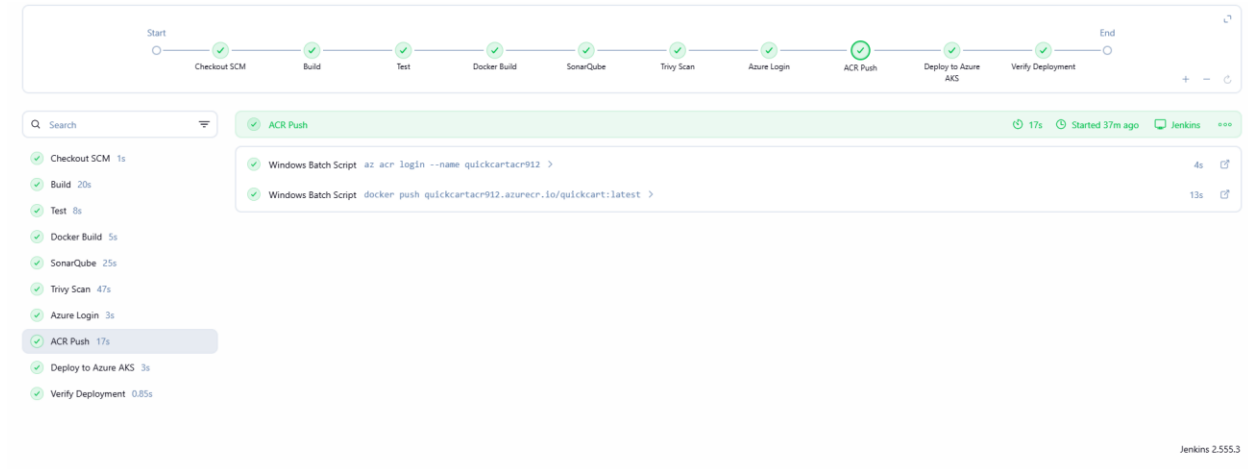
Azure Container Registry (ACR) was used as the private container registry for the QuickCart application.

The Jenkins pipeline builds the Docker image, performs a Trivy security scan, authenticates with Azure, and pushes the validated image to:

`quickcartacr912.azurecr.io/quickcart:latest`

ACR provides a centralized location for storing the container image before it is deployed to Azure Kubernetes Service.

Figure 8: QuickCart ACR Push through Jenkins



10. Azure Kubernetes Service (AKS)

Azure Kubernetes Service (AKS) was used to host and run the containerized QuickCart application.

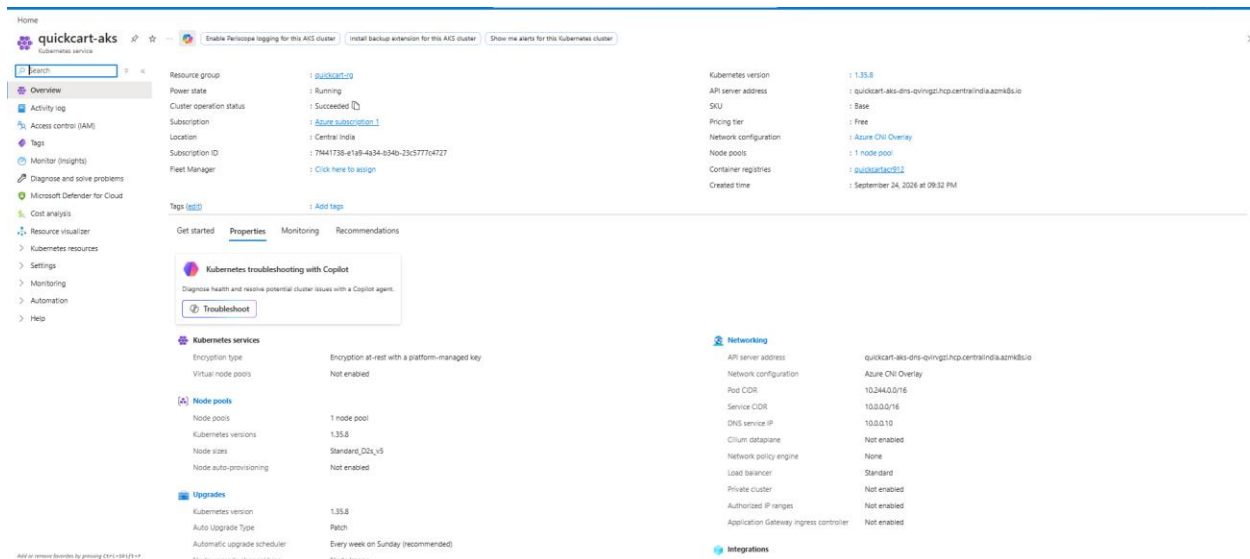
The AKS cluster provides the Kubernetes environment required to deploy, manage, and scale the application containers.

The QuickCart deployment uses:

- **Cluster:** quickcart-aks
- **Resource Group:** quickcart-rg
- **Region:** Central India
- **Container Registry:** quickcartacr912
- **Kubernetes Deployment:** quickcart
- **Replicas:** 1
- **Service Type:** LoadBalancer

The Docker image stored in Azure Container Registry is pulled by AKS and deployed as a Kubernetes Pod.

Figure 9: QuickCart Azure Kubernetes Service Cluster



11. Kubernetes Deployment

The QuickCart application was deployed to the AKS cluster using a Kubernetes Deployment.

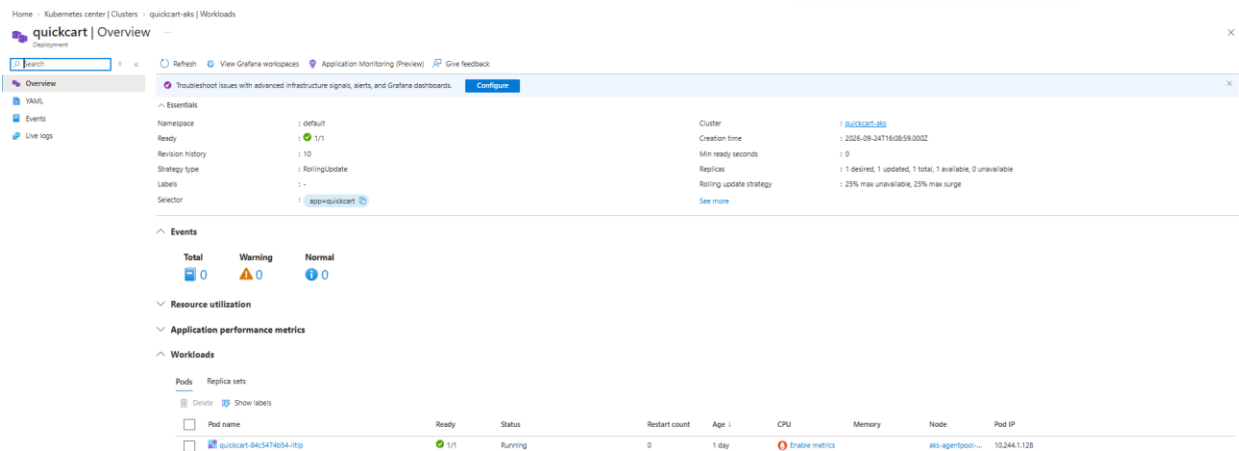
The Deployment manages the QuickCart Pod and ensures that the desired number of application instances are running.

The deployment configuration uses:

- **Deployment:** quickcart
- **Replicas:** 1
- **Container:** quickcart
- **Container Port:** 8081
- **Image:** quickcartacr912.azurecr.io/quickcart:latest

The deployment was successfully created and the QuickCart workload reached the **Ready** state.

Figure 10: QuickCart Kubernetes Deployment on AKS



12. Kubernetes Service and Load Balancer

A Kubernetes LoadBalancer Service was created to expose the QuickCart application outside the AKS cluster.

The Service forwards incoming traffic on port **8081** to the QuickCart application running inside the Kubernetes Pod.

Configuration:

- **Service:** quickcart
- **Type:** LoadBalancer
- **Port:** 8081
- **Target Port:** 8081
- **External IP:** 4.247.239.65

The assigned external IP allows users to access the QuickCart REST API from outside the Kubernetes cluster.

Figure 11: QuickCart Kubernetes LoadBalancer Service

Home > quickcart-aks | Services and ingresses

quickcart Overview

Search

Refresh Add ingress Give feedback

Overview

YAML Events

Essentials

Namespace: default

Labels: {}

Selector: app=quickcart

External traffic policy: Cluster

See more

Cluster: 80000000-0000

Creation time: 2020-09-24T16:12:59.000Z

Type: LoadBalancer

Cluster IP: 10.0.117.237

External IP: 6.247.239.65

Session affinity: None

Ports

Name	Node port	Port	Protocol	Target port	Endpoints
*	30879	8081	TCP	8081	10.244.1.128:8081

13. Final Application Verification

After deploying QuickCart to Azure Kubernetes Service, the application was accessed through the public Azure LoadBalancer endpoint.

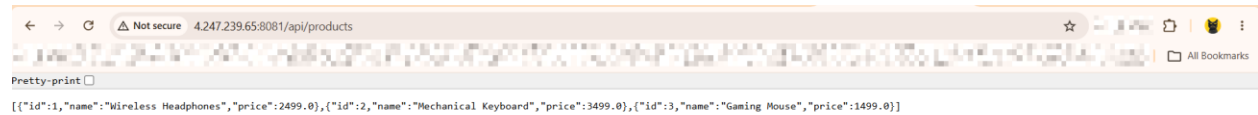
The Product API was tested using:

GET <http://4.247.239.65:8081/api/products>

The API successfully returned the available QuickCart products, confirming that the complete deployment was functioning correctly.

This verifies the complete flow from the container image stored in Azure Container Registry to the running Kubernetes workload and publicly accessible application.

Figure 12: QuickCart Application Running on Azure AKS



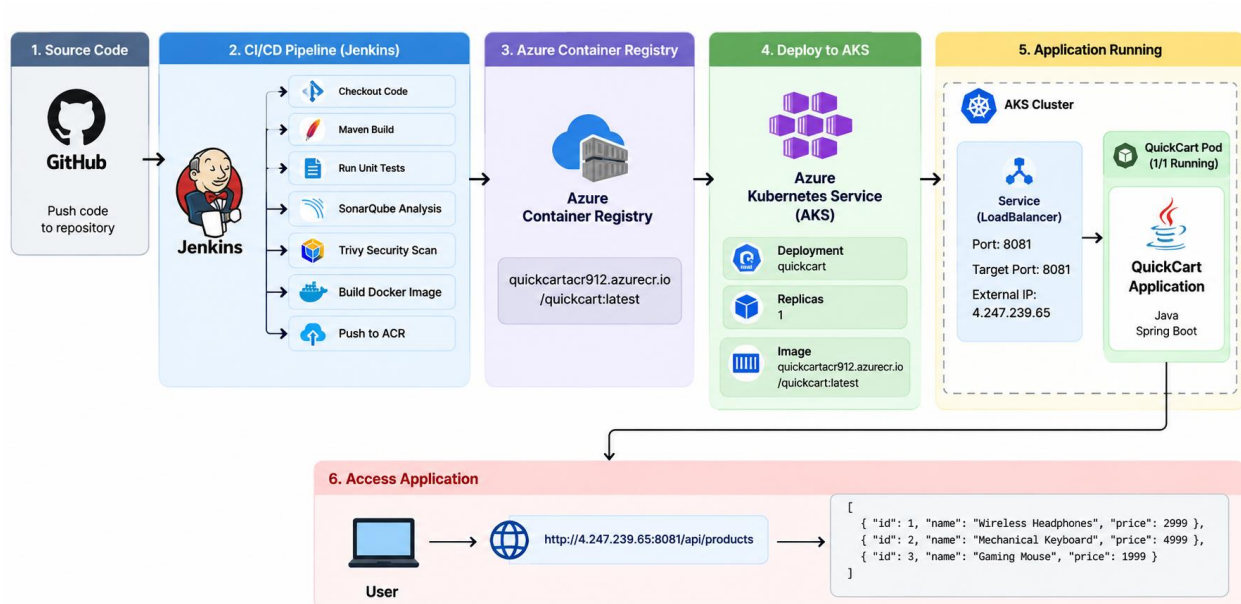
14. End-to-End DevOps Architecture

The QuickCart project implements an end-to-end DevOps workflow that automates the process from source code management to application deployment on Azure Kubernetes Service.

The workflow is:

GitHub → Jenkins → Maven → SonarQube → Trivy → Azure Container Registry → Azure Kubernetes Service → LoadBalancer → QuickCart API

The CI/CD pipeline automatically builds and tests the application, performs code quality analysis and container security scanning, pushes the validated image to Azure Container Registry, and deploys the application to AKS.



15. Challenges and Solutions

Docker Image Security

Initial Trivy scans identified vulnerabilities in the application dependencies. The affected dependencies were updated and the image was rescanned until no HIGH or CRITICAL vulnerabilities remained.

SonarQube Quality Gate

SonarQube was integrated into the Jenkins pipeline to ensure that the application passed the configured quality checks before deployment.

Azure Authentication

A Microsoft Azure Service Principal was configured for Jenkins to securely authenticate with Azure and perform the required deployment operations.

Container Registry Integration

The pipeline was initially configured with DockerHub. It was later integrated with Azure Container Registry so that the Docker image is stored in ACR before deployment to AKS.

Kubernetes Deployment

The application was deployed to AKS using Kubernetes Deployment and Service manifests. A LoadBalancer Service was configured to provide external access to the application.

16. Future Improvements

The QuickCart DevOps implementation can be further enhanced with:

- Prometheus and Grafana for monitoring and visualization.
- Automated alerting for application and infrastructure issues.
- Terraform or Bicep for Infrastructure as Code.
- HTTPS using a custom domain and TLS certificates.
- Horizontal Pod Autoscaling for dynamic scaling.
- Automated rollback strategies for failed deployments.
- Enhanced centralized logging.

17. Conclusion

QuickCart demonstrates a complete end-to-end DevOps implementation using modern development, security, CI/CD, containerization, and cloud-native technologies.

The project successfully automates the journey from source code in GitHub to a containerized application running on Azure Kubernetes Service.

The final implementation integrates **GitHub, Jenkins, Maven, SonarQube, Trivy, Docker, Azure Container Registry, Kubernetes, and Azure AKS**, providing a practical demonstration of CI/CD automation and cloud deployment.